

```
> initializing documentation system... [OK]
> loading workflow modules... [OK]
> rendering cover page... [OK]
```

=====

Arduino and Assembly Files

Using Termux and ArduinoDroid

Termux + Neovim + AVR + ArduinoDroid

Arduino (.ino) & AVR Assembly (.asm)
from phone to microcontroller -- no PC needed

=====

BEGINNER FRIENDLY

STEP-BY-STEP

PHONE-ONLY WORKFLOW

OS	Termux (Android) + Ubuntu proot
Editor	Neovim
Compiler	arduino-cli / avra
Target	Arduino UNO (ATmega328P)
Uploader	ArduinoDroid via OTG
Covers	C/C++ (.ino) and AVR Assembly (.asm)

```
> MOBILE EMBEDDED DEVELOPMENT GUIDE v1.0
> Copyright (c) 2026 IITH-SRFP
> Target Hardware : Arduino UNO (ATmega328P @ 16MHz)
> Chapter 1 : Arduino C/C++ Workflow (.ino)
> Chapter 2 : AVR Assembly Workflow (.asm)
> Prerequisite : Android phone, OTG cable, Arduino UNO
> No PC required. Everything runs on your phone.
> Press any key to continue...
> [loading table of contents]
```

-- What you will learn --

- [1] Setting up a complete embedded dev environment on Android
- [2] Writing Arduino sketches in Neovim
- [3] Compiling with arduino-cli inside Ubuntu (proot)
- [4] Writing AVR Assembly in Neovim
- [5] Assembling with avra inside Termux
- [6] Uploading .hex files to Arduino UNO via ArduinoDroid + OTG
- [7] Debugging common errors

Contents

1	Arduino C/C++ Workflow	4
1.1	Software Required	4
1.2	Folder Structure	4
1.3	Step-by-Step Guide	5
1.3.1	STEP 1 -- Grant Storage Permission	5
1.3.2	STEP 2 -- Create Project Folder	5
1.3.3	STEP 3 -- Create the Sketch File	6
1.3.4	STEP 4 -- Open Ubuntu (proot)	6
1.3.5	STEP 5 -- Navigate to Project	6
1.3.6	STEP 6 -- Edit the Sketch in Neovim	6
1.3.7	STEP 7 -- Compile with arduino-cli	7
1.3.8	STEP 8 -- Find the HEX File	8
1.3.9	STEP 9 -- Exit Ubuntu	8
1.3.10	STEP 10 -- Copy HEX to Android Storage	8
1.3.11	STEP 11 -- Upload via ArduinoDroid	9
1.4	Quick Reference -- Daily Workflow	9
1.5	Automation: build.sh Script	10
1.6	Common Errors and Fixes	10
2	AVR Assembly Workflow	12
2.1	Software Required	12
2.2	AVR Assembly Basics	13
2.2.1	Registers	13
2.2.2	I/O Registers	13
2.2.3	Essential Instructions	14
2.3	Step-by-Step Guide	14
2.3.1	STEP 1 -- Grant Storage Permission	14
2.3.2	STEP 2 -- Create Project Folder	14
2.3.3	STEP 3 -- Create an Assembly File	14
2.3.4	STEP 4 -- Edit in Neovim	14
2.3.5	STEP 5 -- Assemble with avra	16
2.3.6	STEP 6 -- Verify Output Files	16
2.3.7	STEP 7 -- Copy HEX to Android Storage	16
2.3.8	STEP 8 -- Upload via ArduinoDroid	17

2.4 Quick Reference -- Daily Workflow	17
2.5 Useful Daily Commands	18
2.6 Common Errors and Fixes	18
2.7 Folder Structure for Multiple Projects	19
3 Side-by-Side Comparison	20
3.1 Workflow Comparison	20
3.2 The Complete Dev Stack	21

Arduino C/C++ Workflow

> workflow: Termux → Ubuntu (proot) → arduino-cli → ArduinoDroid

This chapter walks you through every step needed to write, compile, and upload an Arduino sketch (.ino) using **only your Android phone**. No PC, no Arduino IDE on a laptop — just Termux, Ubuntu, Neovim, and ArduinoDroid.

§1.1 Software Required

Before you begin, make sure all of these are installed and working on your phone:

#	Software	Purpose
1	Termux	Android terminal emulator — your main shell
2	Ubuntu (proot-distro)	Full Linux environment inside Termux
3	Neovim	Terminal text editor for writing code
4	arduino-cli	Command-line Arduino compiler (inside Ubuntu)
5	Ranger	File manager inside terminal
6	ArduinoDroid	Android app that uploads HEX to Arduino via OTG
7	OTG cable	Connects your phone to the Arduino UNO via USB
8	Arduino UNO	Target hardware

§1.2 Folder Structure

Before running any commands, understand the two-location folder layout:

```

Phone Storage (Android)
|
+-- Arduino/
|   +-- my_project/
|       +-- my_project.ino
|
+-- ArduinoHEX/
|   +-- my_project.ino.hex

Ubuntu (proot) -- internal
~/arduino/
|
+-- my_project/
|   +-- my_project.ino
|   +-- my_project.ino.hex
|   +-- build/ (auto-generated)

```

[NOTE: Sketch Name Rule]

The .ino filename must match the folder name exactly.
 Wrong: mycode/sketch.ino
 Correct: mycode/mycode.ino
 Arduino-cli throws a ‘main file missing’ error otherwise.

§1.3 Step-by-Step Guide

1.3.1 STEP 1 -- Grant Storage Permission

Open Termux for the very first time and run:

```
termux-setup-storage
```

A popup will appear asking for storage permission. Tap **Allow**. This only needs to be done once. After this, your Android storage is accessible at ~/storage/shared/.

[!] If you skip this step

You will not be able to copy files between Ubuntu and Android storage.
 The cp command to ~/storage/shared/ will fail with ‘Permission denied’.

1.3.2 STEP 2 -- Create Project Folder

Inside Termux, create the project directory in Android shared storage:

```
mkdir -p ~/storage/shared/Arduino/my_project
cd ~/storage/shared/Arduino/my_project
```

Replace `my_project` with your actual project name — e.g. `7447_project`, `blink`, `motor_ctrl`.

1.3.3 STEP 3 -- Create the Sketch File

Open Ranger (terminal file manager):

```
ranger
```

Inside Ranger, press `:touch my_project.ino` and hit Enter to create the file. Then press `q` to quit Ranger.

```
[#] Alternatively -- create directly in shell
```

```
touch my_project.ino
This creates an empty file instantly without opening Ranger.
```

1.3.4 STEP 4 -- Open Ubuntu (proot)

```
proot-distro login ubuntu
```

You are now inside a full Ubuntu Linux environment. Your prompt changes to something like `root@localhost:~#`. Everything from here until “exit” runs inside Ubuntu.

1.3.5 STEP 5 -- Navigate to Project

If your project is in Android shared storage:

```
cd /storage/emulated/0/Arduino/my_project
```

If you prefer to keep it inside Ubuntu’s own filesystem:

```
cd ~/arduino/my_project
```

Confirm the `.ino` file exists:

```
ls
```

```
my_project.ino
```

1.3.6 STEP 6 -- Edit the Sketch in Neovim

```
nvim my_project.ino
```

Inside Neovim, write your Arduino sketch. Below is a minimal example to test the full pipeline — it blinks the built-in LED on pin 13:

Listing 1.1. Minimal blink sketch — my_project.ino

```

1 // Blink built-in LED (pin 13) every 500ms
2 // Tests that the entire workflow compiles and uploads
  correctly
3
4 void setup() {
5     pinMode(13, OUTPUT);    // set pin 13 as output
6 }
7
8 void loop() {
9     digitalWrite(13, HIGH); // LED on
10    delay(500);              // wait 500ms
11    digitalWrite(13, LOW);   // LED off
12    delay(500);              // wait 500ms
13 }

```

Neovim essential commands:

Key / Command	Action
i	Enter insert mode (start typing)
Esc	Exit insert mode, return to normal mode
:w	Save the file
:q	Quit (only works if no unsaved changes)
:wq	Save and quit
:q!	Force quit without saving
dd	Delete current line
u	Undo last change
Ctrl+r	Redo
/text	Search for “text” in the file

1.3.7 STEP 7 -- Compile with arduino-cli

```
arduino-cli compile --fqbn arduino:avr:uno --export-binaries .
```

Breaking down this command:

Part	Meaning
arduino-cli	The Arduino command-line compiler
compile	Tells it to build/compile
-fqbn arduino:avr:uno	Fully Qualified Board Name — targets Arduino UNO
-export-binaries	Saves the .hex output file alongside the sketch
.	Compile the sketch in the current directory

[✓] Expected successful output

Sketch uses 924 bytes (2%) of program storage space.
Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory.

[!] If you see errors

Error: main file missing -- your sketch name doesn't match the folder name.
undefined reference to setup() -- setup() function is missing or has a typo.
undefined reference to loop() -- loop() function is missing.

1.3.8 STEP 8 -- Find the HEX File

```
find . -name "*.hex"
```

```
./build/arduino.avr.uno/my_project.ino.hex
```

The .hex file is the compiled binary ready to be flashed onto the Arduino.

1.3.9 STEP 9 -- Exit Ubuntu

```
exit
```

You are now back in Termux.

1.3.10 STEP 10 -- Copy HEX to Android Storage

```
cp /data/data/com.termux/files/usr/var/lib/proot-distro/\
containers/ubuntu/rootfs/root/arduino/my_project/\
build/arduino.avr.uno/my_project.ino.hex \
~/storage/shared/ArduinoHEX/
```

[Path Note]

The long path above is the Ubuntu filesystem root accessible from Termux. Replace `my_project` with your actual project name throughout.

Verify:

```
ls ~/storage/shared/ArduinoHEX
```

```
my_project.ino.hex
```

1.3.11 STEP 11 -- Upload via ArduinoDroid

1. Connect your **phone** to the **OTG adapter**
2. Connect the **OTG adapter** to the **Arduino UNO** via USB cable
3. Open **ArduinoDroid**
4. Navigate to Internal Storage / ArduinoHEX
5. Tap `my_project.ino.hex`
6. Tap **Upload**
7. Wait for Done confirmation



§1.4 Quick Reference -- Daily Workflow

Once the environment is set up, your daily cycle is just these commands:

Listing 1.2. Daily workflow — Arduino C/C++

```
# --- Inside Termux ---
proot-distro login ubuntu

# --- Inside Ubuntu ---
cd ~/arduino/my_project
nvim my_project.ino           # edit code
arduino-cli compile --fqbn arduino:avr:uno --export-binaries .
exit

# --- Back in Termux ---
cp /data/data/com.termux/files/usr/var/lib/proot-distro/\
containers/ubuntu/rootfs/root/arduino/my_project/\
```

```
build/arduino.avr.uno/my_project.ino.hex \
~/storage/shared/ArduinoHEX/

# --- Open ArduinoDroid and upload ---
```

§1.5 Automation: build.sh Script

Typing the long cp command every time is tedious. Create a shell script to do it in one shot:

Listing 1.3. build.sh — automate compile + copy

```
#!/bin/bash
# build.sh --- compile and copy HEX in one command
# Usage: ./build.sh my_project

PROJECT=$1

echo "[*] Compiling $PROJECT..."
arduino-cli compile --fqbn arduino:avr:uno --export-binaries .

if [ $? -eq 0 ]; then
    echo "[OK] Compilation successful"
    echo "[*] Copying HEX to ArduinoHEX..."
    cp ./build/arduino.avr.uno/${PROJECT}.ino.hex \
      ~/storage/shared/ArduinoHEX/
    echo "[OK] Done! Open ArduinoDroid and upload."
else
    echo "[!] Compilation failed. Fix errors above."
fi
```

Make it executable and run:

```
chmod +x build.sh
./build.sh my_project
```

Now your daily workflow collapses to:

```
nvim my_project.ino
./build.sh my_project
# -> open ArduinoDroid, upload
```

§1.6 Common Errors and Fixes

Error Message	Cause	Fix
main file missing	Sketch name doesn't match folder name	Rename .ino to match folder or vice versa
undefined reference to setup()	setup() missing or typo	Check code structure and braces
undefined reference to loop()	loop() missing	Add a valid loop() function
nt does not name a type	Typo: nt instead of int	Correct the typo
No such file or directory	Wrong path used	Run find . -name "*.hex" to locate
Not a directory	Destination is a file not a folder	rm the file, then mkdir
No boards found	USB / permission issue	Use ArduinoDroid for upload
Permission denied	Storage permission not granted	Re-run termux-setup-storage

AVR Assembly Workflow

> workflow: Termux → avra → ArduinoDroid (no Ubuntu needed)

AVR Assembly gives you **complete, direct control** over the ATmega328P hardware. Every instruction you write maps 1-to-1 to a CPU operation — no abstractions, no framework overhead. This chapter shows how to write, assemble, and upload AVR Assembly code entirely from Termux on your phone.

[Assembly vs Arduino C/C++]

Arduino C/C++ is easier and faster to write.
AVR Assembly is harder but gives you full hardware control.
Use C/C++ for projects. Use Assembly to learn the hardware deeply.

§2.1 Software Required

#	Software	Purpose
1	Termux	Main terminal environment
2	avra	AVR Assembler — converts .asm to .hex
3	Neovim	Write and edit .asm files
4	ArduinoDroid	Upload .hex to Arduino via OTG
5	OTG cable	Connects phone to Arduino UNO
6	Arduino UNO	Target hardware

[No Ubuntu Needed Here]

Unlike the Arduino C/C++ workflow, AVR Assembly uses avra which runs directly in Termux -- no need to enter Ubuntu (proot). Simpler!

§2.2 AVR Assembly Basics

Before writing code, understand the key concepts of AVR Assembly:

2.2.1 Registers

The ATmega328P has **32 general-purpose 8-bit registers** named r0 through r31.

Register(s)	Common Use
r0 - r15	Lower registers; limited instruction support
r16 - r31	Upper registers; used for most operations
r26:r27 (X)	16-bit pointer register X
r28:r29 (Y)	16-bit pointer register Y
r30:r31 (Z)	16-bit pointer register Z; used for lpm

[#] Always use r16 and above

Instructions like ldi (load immediate) only work with registers r16-r31. Using r0-r15 with ldi will throw an assembler error.

2.2.2 I/O Registers

The ATmega328P controls hardware through special I/O registers:

Register	Name	Purpose
DDRx	Data Direction	Set each pin as INPUT (0) or OUTPUT (1)
PORTx	Port Output	Write HIGH (1) or LOW (0) to output pins
PINx	Port Input	Read current state of input pins

Where x is B, C, or D for the three I/O ports.

2.2.3 Essential Instructions

Instruction	What it does
<code>ldi Rd, K</code>	Load Immediate: put constant K into register Rd
<code>out A, Rd</code>	Write register Rd to I/O address A
<code>in Rd, A</code>	Read I/O address A into register Rd
<code>rjmp label</code>	Relative Jump: jump to label (loops)
<code>sbis A, b</code>	Skip if Bit in I/O register is Set
<code>sbic A, b</code>	Skip if Bit in I/O register is Cleared
<code>sbi A, b</code>	Set Bit b in I/O register A
<code>cbi A, b</code>	Clear Bit b in I/O register A
<code>nop</code>	No Operation: does nothing for 1 cycle
<code>call label</code>	Call subroutine at label
<code>ret</code>	Return from subroutine

§2.3 Step-by-Step Guide

2.3.1 STEP 1 -- Grant Storage Permission

```
termux-setup-storage
```

(Only required once. Skip if already done.)

2.3.2 STEP 2 -- Create Project Folder

```
mkdir -p ~/assembly
cd ~/assembly
```

All your `.asm` files live here.

2.3.3 STEP 3 -- Create an Assembly File

```
touch blink.asm
```

2.3.4 STEP 4 -- Edit in Neovim

```
nvim blink.asm
```

Below is a complete, well-commented AVR Assembly program that blinks the LED on Port B, pin 5 (Arduino pin 13):

Listing 2.1. blink.asm — LED blink in AVR Assembly

```

1  ;

2  ; blink.asm --- Blink LED on PB5 (Arduino UNO pin 13)
3  ; Target: ATmega328P @ 16 MHz
4  ;

5
6  .include "m328pdef.inc"      ; ATmega328P register definitions
7
8  .org 0x0000                  ; start execution at reset vector
9
10 ;          Setup

11     ldi r16, 0xFF             ; load 0xFF (all 1s) into r16
12     out DDRB, r16             ; set all Port B pins as OUTPUT
13
14 ;          Main Loop

15 main_loop:
16     ldi r16, 0b00100000      ; bitmask for PB5 (pin 13)
17     out PORTB, r16           ; set PB5 HIGH -> LED ON
18     call delay_long           ; wait ~500ms
19
20     ldi r16, 0b00000000      ; all bits 0
21     out PORTB, r16           ; set PB5 LOW -> LED OFF
22     call delay_long           ; wait ~500ms
23
24     rjmp main_loop           ; loop forever
25
26 ;          Delay Subroutine (~500ms @ 16MHz)

27 delay_long:
28     ldi r17, 30               ; outer counter
29 outer_loop:
30     ldi r18, 0                ; middle counter (256 iterations)
31 middle_loop:
32     ldi r19, 0                ; inner counter (256 iterations)
33 inner_loop:
34     nop                       ; 1 clock cycle wasted
35     dec r19                   ; decrement inner counter
36     brne inner_loop           ; loop until r19 == 0
37     dec r18
38     brne middle_loop
39     dec r17
40     brne outer_loop

```


41

`ret` ; return from subroutine

2.3.5 STEP 5 -- Assemble with avra

```
avra blink.asm
```

What avra does:



[✓] Successful assembly output

```

AVRA: advanced AVR macro assembler Version 1.4.2
Pass 1...
Pass 2...
done
Used 42 words (84 bytes) of program storage

```

[!] If avra says "can't find m328pdef.inc"

```

The include file for the ATmega328P register names is missing.
Fix: apt-get install avr-libc (inside Ubuntu)
or copy m328pdef.inc into your project folder.

```

2.3.6 STEP 6 -- Verify Output Files

```
ls ~/assembly
```

```

blink.asm
blink.hex      <- this is what you upload
blink.eep.hex  <- EEPROM data (usually empty)
blink.obj      <- object file (intermediate)
blink.cof      <- COFF debug info

```

You only need blink.hex. The others can be deleted:

```
rm *.obj *.cof *.eep.hex
```

2.3.7 STEP 7 -- Copy HEX to Android Storage

```
cp blink.hex ~/storage/shared/ArduinoHEX/
```

Verify:

```
ls ~/storage/shared/ArduinoHEX
```

```
blink.hex
```

2.3.8 STEP 8 -- Upload via ArduinoDroid

1. Plug OTG adapter into phone
2. Connect Arduino UNO to OTG adapter via USB cable
3. Open **ArduinoDroid**
4. Go to Internal Storage / ArduinoHEX
5. Select `blink.hex`
6. Tap **Upload**
7. You should see the TX/RX LEDs blink during upload
8. Once done: the LED on pin 13 should start blinking!

§2.4 Quick Reference -- Daily Workflow

Listing 2.2. Daily workflow — AVR Assembly

```
# All inside Termux --- no Ubuntu needed

cd ~/assembly

nvim blink.asm          # write/edit code

avra blink.asm          # assemble -> generates blink.hex

cp blink.hex ~/storage/shared/ArduinoHEX/

# Open ArduinoDroid -> upload blink.hex
```

§2.5 Useful Daily Commands

Command	What it does
<code>nvim file.asm</code>	Open file in Neovim editor
<code>avra file.asm</code>	Assemble and generate <code>.hex</code>
<code>ls</code>	List files in current directory
<code>cp file.hex</code> <code>~/storage/shared/ArduinoHEX/</code>	Copy HEX to phone storage
<code>rm *.obj *.cof *.eep.hex</code>	Delete intermediate files
<code>avra timer.asm</code>	Assemble a different project
<code>cat file.hex</code>	Print HEX contents (inspect)
<code>wc -l file.asm</code>	Count lines of assembly code

§2.6 Common Errors and Fixes

Error	Cause	Fix
<code>avra: can't open file</code>	Wrong filename or path	Run <code>ls</code> and check exact name
<code>can't find m328pdef.inc</code>	Include file missing	Copy <code>m328pdef.inc</code> to project folder
Error at line N	Syntax error in code	Check commas, labels, instruction spelling
<code>ldi</code> only works <code>r16-r31</code>	Used <code>ldi</code> on <code>r0-r15</code>	Change register to <code>r16</code> or higher
Upload fails	USB/OTG issue	Check cable, OTG adapter, board selection
LED not blinking	Code logic error	Check <code>DDRB</code> setup and <code>PORTB</code> bitmask
undefined symbol	Label typo or missing	Check all labels are defined before use

§2.7 Folder Structure for Multiple Projects

Side-by-Side Comparison

> when to use which workflow

§3.1 Workflow Comparison

Aspect	Arduino C/C++ (.ino)	AVR Assembly (.asm)
Language	C++ with Arduino Framework	AVR Assembly
Compiler/Assembler	arduino-cli	avra
Runs in	Ubuntu (proot)	Termux directly
Output	.hex	.hex + .obj + .cof
Editor	Neovim	Neovim
Upload	ArduinoDroid	ArduinoDroid
Difficulty	Beginner-friendly	Intermediate
Abstraction	High (Arduino hides hardware)	None (full hardware control)
Flash usage	More (framework overhead)	Minimal
When to use	General projects	Hardware-level learning

§3.2 The Complete Dev Stack



=====

You now have a complete embedded development ecosystem on your phone.

No PC. No Arduino IDE. Just Termux, Neovim, and ArduinoDroid.

=====